

The Dedekind Topos: A Structuralist Mereology in C++23

The Dedekind Authors

March 27, 2026

Abstract

Your abstract text.

1 Introduction

The quest for a unified computational environment that simultaneously satisfies the rigour of formal proof assistants and the raw efficiency of hardware-level numerics remains a central challenge in computer science. Historically, this has been a binary choice: one either employs a dedicated prover such as Coq or Lean to ensure mathematical correctness at the cost of execution speed, or one utilizes traditional numeric libraries that prioritize throughput while treating mathematical axioms as mere documentation.

In this paper, we propose **dedekind**, a Domain Specific Language (DSL) embedded in modern C++23, designed to bridge this dichotomy. Our approach is rooted in the *Curry-Howard correspondence*, which establishes a direct isomorphism between computer programs and mathematical proofs. By leveraging C++23 **concepts** and template metaprogramming, we treat the C++ type system as a logic engine. While the Clang compiler may not be the most sophisticated theorem prover in existence, its ubiquity and industry-standard optimization passes make it perhaps the most accessible vehicle for what we term *Computational Structuralism*.

Following the intuition provided by Wadler in *Theorems for Free*, **dedekind** utilizes parametric polymorphism to derive structural properties of mathematical objects—such as associativity, commutativity, and idempotency—directly from their type signatures. By encoding mereotopological and category-theoretic axioms into the "structural registry" of the C++ translation unit, we enable the compiler to perform symbolic pruning of the computational Directed Acyclic Graph (DAG) before a single machine instruction is emitted. The resulting binary is as lean as a manually optimized "naked" loop, yet remains as rigorous as a formal proof, reifying the DAG as a concrete artifact of the underlying mathematical theorem.

2 Methods

The methodology of **dedekind** is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next.

2.1 The Unified Dedekind Ontology

The **dedekind.ontology** module serves as a central registry of structural truths. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. By adopting a *Structuralist* posture—viewing mathematical objects as positions within a structure rather than isolated collections—we facilitate zero-overhead formal verification within the C++ type system.

The ontology is organized into four primary levels of abstraction, mirroring the historical construction of mathematics:

Level -1 to 0: Foundations (Bricks and Cement) At the base lies `:logic`, defining the subobject classifier Ω and predicate logic. This is supported by `:species`, which provides reified machine primitives, and `:category`, which establishes the "Highways" of the system via Morphisms, Functors, and Kleisli Triples.

Level 1: Space (Body and Relation) The `:mereology` partition defines the rules of presence (parts and wholes), while `:order` and `:topology` establish the rules of relation and closeness (posets, lattices, and open sets).

Level 2: Scale (Measurement and Path) The `:sequences` module maps magnitudes to enumerations, providing the necessary machinery for limits and convergence.

Level 3: Soul (Harmony and Action) This level implements the pure algebraic laws (`:algebra`) including Groups, Rings, and Fields, alongside their realized `:morphologies` (Cyclic, Ordered, and Archimedean forms).

Level 4: Realization (The Registry) The final level, `:geometry` and `:numbers`, realizes the continuum through the species registry, formally constructing the inclusion $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$.

2.2 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

Table 1: The Dedekind Semantic Mapping (Expanded)

Mathematical Concept	Symbol	C++ Operator / Type	C++ Concept
<i>Mereology</i>			
Parthood (Pre-order)	$\sqsubseteq$	<code>operator<=</code>	<code>IsPartOf</code>
Mereological Sum (Join)	$\sqcup$	<code>operator </code>	<code>IsJoinSemilattice</code>
Mereological Product (Meet)	$\sqcap$	<code>operator&</code>	<code>IsMeetSemilattice</code>
Universal Complement	$\neg$	<code>operator!</code>	<code>IsComplemented</code>
Initial Object (Empty)	$\emptyset$	<code>0<T></code>	<code>IsInitialObject</code>
<i>Category Theory</i>			
Characteristic Morphism	χ	<code>operator()</code>	<code>IsSet / IsProperPart</code>
Monadic Bind (Push)	$\gg=$	<code>operator>=</code>	<code>IsMonad</code>
Kleisli Composition	$\circ_K$	<code>operator></code>	<code>IsMonad</code>
Comonadic Extend (Pull)	$\ll=$	<code>operator<=</code>	<code>IsComonad</code>
Co-Kleisli Composition	$\circ_{CoK}$	<code>operator<</code>	<code>IsComonad</code>
Counit (Extraction)	ε	<code>extract<F></code>	<code>IsComonad</code>
<i>Algebra</i>			
Equivalence / Identity	$\cong$	<code>operator==</code>	<code>IsExtensional</code>
Associativity	$(a \circ b) \circ c$	<code>is_associative_v</code>	<code>IsSemigroupoid</code>
Idempotency	$a \circ a = a$	<code>is_idempotent_v</code>	<code>IsIdempotent</code>